

UNIT 2 - LARAVEL FUNDAMENTALS

Comprehensive Study & Reference Guide

1. Blade Templates for Dynamic Views

2. Laravel Migrations

3. Database Configuration

4. CRUD with Eloquent ORM

1. Working with Blade Templates

Laravel's powerful, lightweight templating engine

Extension

`.blade.php`

Location

`resources/views/`

Compilation

Compiled to PHP, auto-cached

Output (escaped)

`{{ $var }}`

Raw HTML output

`{!! $var !!}`

Directives

`@if, @foreach, @extends...`

Blade compiles down to plain PHP and caches until modified — expressive syntax without sacrificing performance.

Blade Directives Quick Reference

@extends('layout')

Inherit from a parent layout

@section('name')

Define a content section

@yield('name')

Output a section in the layout

@include('partial')

Include another view file

@push / @stack

Push scripts/styles to a named stack

@csrf

Generate a CSRF token field

@if / @elseif / @else

Conditional statements

@foreach / @forelse

Loop over arrays/collections

@auth / @guest

Check authentication status

@method('PUT')

Spoof HTTP method in forms

@error('field')

Display validation errors

@php ... @endphp

Inline PHP block

Template Inheritance & Components

Template Inheritance Flow

Master Layout

layouts/app.blade.php



@yield('content')

Placeholder in master



Child View

@extends + @section



Rendered Page

Combined HTML output

Components

```
php artisan make:component Alert
```

```
php artisan make:component Forms/Input
```

```
@props(['type' => 'info'])
```

```
<x-alert type='success'>
```

```
$attributes->merge(['class' => ...])
```

Slots

Named regions inside components. Use `{{ $slot }}` for default, or `$attributes` for passing classes, IDs, and other HTML attributes through.

2. Laravel Migrations

Version control for your database schema

Why Use Migrations?

- Track schema changes in version control (Git)
- Collaborate without sharing SQL dumps
- Rollback schema changes on errors
- Recreate database from scratch automatically
- Seed test data after migration

Artisan Commands

```
php artisan make:migration create_flights_table
```

```
php artisan make:model Product -m
```

```
php artisan make:model Product -mcsf
```

```
php artisan migrate
```

```
php artisan migrate:rollback
```

```
php artisan migrate:fresh --seed
```

```
php artisan migrate:status
```

💡 Migration files live in `database/migrations/`. The timestamp prefix (e.g. `2024_01_15_...`) ensures correct execution order.

Migration Structure & Column Types

Common Column Types

<code>\$table->id()</code>	Auto-increment primary key
<code>\$table->string('name')</code>	VARCHAR(255)
<code>\$table->text('bio')</code>	TEXT field
<code>\$table->integer('qty')</code>	INTEGER field
<code>\$table->decimal('price', 10, 2)</code>	DECIMAL(10,2)
<code>\$table->boolean('active')</code>	BOOLEAN / TINYINT
<code>\$table->timestamps()</code>	created_at & updated_at
<code>\$table->softDeletes()</code>	deleted_at for soft delete
<code>->nullable()</code>	Allow NULL values
<code>->default(val)</code>	Set default value
<code>->foreignId('cat_id')->constrained()</code>	FK with constraint

Migration File Structure

```
return new class extends Migration {

    // Runs when migrating
    public function up(): void {
        Schema::create('products',
            function (Blueprint $table) {
                $table->id();
                $table->string('name');
                $table->decimal('price',10,2);
                $table->timestamps();
            });
    }

    // Runs on rollback
    public function down(): void {
        Schema::dropIfExists('products');
    }
};
```

3. Database Configuration

MySQL

PostgreSQL

SQLite

SQL Server

.env Configuration

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=my_app
DB_USERNAME=root
DB_PASSWORD=secret
```

Raw DB Queries

```
// SELECT
$users = DB::select(
    'select * from users where active=?',
    [1]
);

// INSERT
DB::insert('insert into users
(id,name) values (?,?)', [1,'Marc']);
```

DB Artisan Tools

php artisan db

Connect to default DB

php artisan db:show

Full DB overview

php artisan db:table users

Inspect specific table

php artisan tinker

Interactive REPL

4. CRUD with Eloquent ORM

Object-Relational Mapper — database tables mapped to PHP classes

CREATE

```
Product::create([...]);

new Product(); $p->save();

Product::firstOrCreate([...]);

Product::updateOrCreate([...]);
```

READ

```
Product::all();

Product::find($id);

Product::findOrFail($id);

Product::where(...)->get();

Product::paginate(15);
```

UPDATE

```
$p->price = 24.99; $p->save();

$p->fill([...])>save();

Product::where()->update([...]);

$p->increment('stock', 50);
```

DELETE

```
$p->delete();

Product::where()->delete();

Product::destroy($id);

$p->restore(); // soft delete

$p->forceDelete();
```

⚠ Mass assignment requires \$fillable array in your model. Use \$casts to auto-convert column types (e.g. 'price' => 'decimal:2', 'is_active' => 'boolean').

Eloquent: Models & Relationships

Model Anatomy

```
class Product extends Model {
    use SoftDeletes;

    protected $table = 'products';

    protected $fillable = [
        'name', 'price', 'stock',
        'category_id', 'is_active'
    ];

    protected $casts = [
        'price'      => 'decimal:2',
        'is_active' => 'boolean',
        'metadata'   => 'array',
    ];

    protected $hidden = ['notes'];
}
```

Relationship Types

One To One

`hasOne (Phone::class)`

User → Phone

One To Many

`hasMany (Comment::class)`

Post → Comments

Belongs To

`belongsTo (User::class)`

Comment → User

Many To Many

`belongsToMany (Role::class)`

User ↔ Roles (pivot)

Eager Loading (avoids N+1):

`Product::with(['category', 'reviews'])->get();`

Key Artisan Commands Reference

```
php artisan make:model Product -m
```

Create model with migration

```
php artisan make:controller
```

```
ProductController -r
```

Resource controller

```
php artisan make:migration create_X_table
```

Create migration file

```
php artisan migrate
```

Run pending migrations

```
php artisan migrate:rollback
```

Undo last migration batch

```
php artisan migrate:fresh --seed
```

Wipe, re-migrate & seed

```
php artisan migrate:status
```

View migration status

```
php artisan tinker
```

Interactive REPL for testing

```
php artisan config:clear
```

Clear config cache

```
php artisan config:cache
```

Re-cache config for production

```
php artisan route:list
```

View all defined routes

```
php artisan make:component Alert
```

Create a Blade component

```
php artisan db:show
```

Show database overview

```
php artisan db:table users
```

Inspect a specific table

Unit 2 Summary

Blade Templates

Directives, inheritance, components, slots, forms

Migrations

Schema version control, column types, up/down lifecycle

Database Config

.env setup, query builder, transactions, multi-connections

Eloquent ORM

Models, CRUD methods, relationships, eager loading